

SERVICIOS DE RED EN UBUNTU SERVER

FTP | IIS | SSH

Administracion y Configuracion de Servicios en Sistemas Linux

Asignatura: Servicios en Red

INDICE DE CONTENIDOS

1. Introduccion a los Servicios de Red
2. Servicio FTP (File Transfer Protocol)
 - 2.1 Que es FTP
 - 2.2 Historia y evolucion
 - 2.3 Modos de conexion: activo y pasivo
 - 2.4 Instalacion de vsftpd en Ubuntu Server
 - 2.5 Configuracion de vsftpd
 - 2.6 Usuarios y permisos
 - 2.7 FTP con TLS/SSL (FTPS)
 - 2.8 Comandos FTP mas importantes
 - 2.9 Alternativas: ProFTPD y Pure-FTPd
3. Servidor Web IIS en Ubuntu (Apache como equivalente)
 - 3.1 Que es IIS y sus equivalentes en Linux
 - 3.2 Instalacion de Apache2
 - 3.3 Configuracion de VirtualHosts
 - 3.4 Modulos de Apache
 - 3.5 HTTPS con Let's Encrypt
 - 3.6 PHP y bases de datos (LAMP)
 - 3.7 Comparativa IIS vs Apache vs Nginx
4. Servicio SSH (Secure Shell)
 - 4.1 Que es SSH
 - 4.2 Historia y versiones del protocolo
 - 4.3 Instalacion y activacion
 - 4.4 Configuracion del servidor SSH
 - 4.5 Autenticacion por clave publica/privada
 - 4.6 Hardening y seguridad SSH
 - 4.7 Tunel SSH y reenvio de puertos
 - 4.8 SCP y SFTP
5. Comparativa de seguridad entre servicios
6. Buenas practicas y recomendaciones

7. Referencias y bibliografia

1. INTRODUCCION A LOS SERVICIOS DE RED

Ubuntu Server es una de las distribuciones Linux mas utilizadas en entornos empresariales y de produccion para la gestion de servicios de red. Su estabilidad, soporte a largo plazo (LTS) y extensa documentacion lo convierten en la plataforma ideal para desplegar servicios como FTP, servidores web y SSH.

Un servicio de red es un programa o conjunto de programas que se ejecutan en un servidor y ofrecen funcionalidades a otros equipos (clientes) a traves de la red. Estos servicios siguen el modelo cliente-servidor y se comunican mediante protocolos estandarizados definidos en documentos RFC (Request for Comments) publicados por la IETF.

Los servicios de red fundamentales en la administracion de sistemas son:

- FTP (File Transfer Protocol): transferencia de archivos entre sistemas.
- SSH (Secure Shell): acceso remoto seguro y transferencia de archivos cifrada.
- HTTP/HTTPS (HyperText Transfer Protocol): servicio web.
- DNS (Domain Name System): resolucion de nombres de dominio.
- DHCP (Dynamic Host Configuration Protocol): asignacion dinamica de IPs.

En este documento nos centraremos en profundidad en FTP, IIS (y su equivalente en Linux, Apache) y SSH, sus instalaciones, configuraciones, parametros de seguridad y buenas practicas.

2. SERVICIO FTP (FILE TRANSFER PROTOCOL)

2.1 Que es FTP

FTP (File Transfer Protocol) es un protocolo de red estandar utilizado para la transferencia de archivos entre un cliente y un servidor a traves de una red TCP/IP. Fue definido originalmente en el RFC 959 en 1985, aunque sus origenes se remontan al RFC 114 de 1971. Es uno de los protocolos de aplicacion mas antiguos de Internet y, aunque ha sido ampliamente sustituido por alternativas seguras como SFTP y FTPS, sigue utilizandose en multitud de entornos.

FTP opera sobre la capa de aplicacion del modelo OSI y utiliza el protocolo de transporte TCP. A diferencia de la mayoría de los protocolos, FTP utiliza dos conexiones TCP separadas:

- Canal de control (puerto 21): se usa para enviar comandos y recibir respuestas entre cliente y servidor. Esta conexion permanece activa durante toda la sesion.
- Canal de datos (puerto 20 en modo activo, o un puerto dinamico en modo pasivo): se usa para la transferencia real de archivos y listados de directorios. Se abre y cierra para cada transferencia.

2.2 Historia y evolucion del protocolo FTP

FTP ha evolucionado considerablemente desde sus origenes. La siguiente tabla resume su historia:

Ano	RFC	Descripcion
1971	RFC 114	Primera especificacion de FTP
1973	RFC 454	Segunda version mejorada
1980	RFC 765	Version revisada con TCP/IP
1985	RFC 959	Especificacion actual y mas usada de FTP
1997	RFC 2228	Extensiones de seguridad para FTP
1998	RFC 2389	Mecanismo de negociacion de caracteristicas
1999	RFC 2428	Soporte para IPv6 y modo pasivo extendido
2007	RFC 4217	Asegurando FTP con TLS

2.3 Modos de conexion: Activo y Pasivo

Modo Activo (PORT)

En el modo activo, el cliente abre un puerto aleatorio mayor que 1023 y notifica al servidor mediante el comando PORT. El servidor entonces inicia la conexion de datos desde su puerto 20 hacia el puerto indicado por el cliente.

Ventaja: simple de configurar en el servidor. Desventaja: los firewalls del cliente frecuentemente bloquean las conexiones entrantes desde el servidor, lo que provoca fallos en la transferencia.

Modo Pasivo (PASV)

En el modo pasivo, es el cliente quien inicia la conexión de datos. El servidor abre un puerto aleatorio y notifica al cliente con el comando PASV. El cliente entonces se conecta a ese puerto para la transferencia.

Ventaja: es compatible con la mayoría de firewalls y NAT, ya que el cliente es quien inicia ambas conexiones. Este modo es el preferido y más usado en la actualidad.

Característica	Modo Activo	Modo Pasivo
Quien inicia datos	Servidor (desde puerto 20)	Cliente
Puerto servidor datos	20	Dinámico (1024-65535)
Compatibilidad NAT	Problemática	Excelente
Compatibilidad firewall	Baja	Alta
Uso recomendado	Redes internas	Internet / producción

2.4 Instalación de vsftpd en Ubuntu Server

vsftpd (Very Secure FTP Daemon) es el servidor FTP más utilizado en distribuciones Linux por su seguridad, rendimiento y facilidad de configuración. Para instalarlo en Ubuntu Server ejecutamos:

```
sudo apt update
sudo apt install vsftpd -y
```

Una vez instalado, verificamos que el servicio está activo:

```
sudo systemctl status vsftpd
sudo systemctl enable vsftpd    # habilitar al inicio del sistema
sudo systemctl start vsftpd     # iniciar el servicio
```

Podemos comprobar que vsftpd está escuchando en el puerto 21:

```
sudo ss -tlnp | grep 21
sudo netstat -tlnp | grep vsftpd
```

2.5 Configuración de vsftpd

El archivo principal de configuración se encuentra en `/etc/vsftpd.conf`. A continuación se detallan los parámetros más importantes:

```
# /etc/vsftpd.conf - Configuración principal
listen=YES                                # vsftpd escucha conexiones IPv4
listen_ipv6=NO                            # deshabilitar IPv6 si no se usa
anonymous_enable=NO                      # IMPORTANTE: deshabilitar acceso
anonimo
local_enable=YES                         # permitir usuarios locales del sistema
write_enable=YES                         # permitir escritura (subida de
archivos)
local_umask=022                           # permisos por defecto de archivos
subidos
dirmessage_enable=YES                    # mostrar mensajes al entrar en
directorios
xferlog_enable=YES                       # habilitar log de transferencias
xferlog_file=/var/log/vsftpd.log
connect_from_port_20=YES                 # usar puerto 20 para datos en modo
activo
chroot_local_user=YES                    # encerrar usuarios en su directorio
home
allow_writeable_chroot=YES               # necesario si el chroot es escribible
secure_chroot_dir=/var/run/vsftpd/empty
pam_service_name=vsftpd
# Modo pasivo
pasv_enable=YES
pasv_min_port=10000
pasv_max_port=10100
pasv_address=IP_PUBLICA_DEL_SERVIDOR
```

Después de modificar el archivo de configuración, siempre hay que reiniciar el servicio:

```
sudo systemctl restart vsftpd
```

2.6 Usuarios y permisos en FTP

La gestión de usuarios es uno de los aspectos más importantes en la administración de un servidor FTP. vsftpd puede usar los usuarios del sistema operativo o una lista de usuarios virtuales.

Creación de un usuario FTP dedicado

```
# Crear usuario sin shell (mayor seguridad)
sudo adduser ftpuser --shell /usr/sbin/nologin
sudo mkdir -p /home/ftpuser/ftp/upload
sudo chown nobody:nogroup /home/ftpuser/ftp
sudo chmod a-w /home/ftpuser/ftp
sudo chown ftpuser:ftpuser /home/ftpuser/ftp/upload
```

Lista de usuarios permitidos

Para restringir el acceso FTP solo a ciertos usuarios, se puede usar el parametro `userlist_enable`:

```
userlist_enable=YES
userlist_file=/etc/vsftpd.userlist
userlist_deny=NO      # la lista es de usuarios PERMITIDOS
```

```
# Agregar usuario a la lista permitida
echo 'ftpuser' | sudo tee -a /etc/vsftpd.userlist
```

2.7 FTP seguro con TLS/SSL (FTPS)

FTP en su forma basica transmite datos en texto plano, incluyendo credenciales. Para solucionar esto, se utiliza FTPS, que cifra la comunicacion mediante TLS/SSL. Esto esta definido en el RFC 4217.

Generacion de certificado autofirmado

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/vsftpd.key \
  -out /etc/ssl/certs/vsftpd.crt
```

Configuracion de TLS en vsftpd

```
# Anadir al final de /etc/vsftpd.conf
ssl_enable=YES
ssl_tlsv1=YES
ssl_sslv2=NO      # deshabilitar SSLv2 (inseguro)
ssl_sslv3=NO      # deshabilitar SSLv3 (inseguro)
rsa_cert_file=/etc/ssl/certs/vsftpd.crt
rsa_private_key_file=/etc/ssl/private/vsftpd.key
require_ssl_reuse=NO
ssl_ciphers=HIGH
force_local_data_ssl=YES      # forzar cifrado en datos
force_local_logins_ssl=YES    # forzar cifrado en login
```

2.8 Comandos FTP mas importantes

Comando	Descripcion
open host	Conectar a un servidor FTP
user nombre	Autenticarse con un usuario
ls / dir	Listar archivos del directorio remoto

cd directorio	Cambiar directorio remoto
lcd directorio	Cambiar directorio local
get archivo	Descargar un archivo del servidor
put archivo	Subir un archivo al servidor
mget *.ext	Descargar multiples archivos
mput *.ext	Subir multiples archivos
delete archivo	Eliminar un archivo remoto
mkdir directorio	Crear un directorio remoto
rmdir directorio	Eliminar un directorio remoto
binary / ascii	Cambiar modo de transferencia
passive	Alternar modo pasivo/activo
bye / quit	Cerrar la conexion FTP

2.9 Alternativas: ProFTPD y Pure-FTPd

ProFTPD

ProFTPD es otro popular servidor FTP para Linux. Su archivo de configuracion es similar en sintaxis a Apache, lo que facilita su administracion a quienes ya conocen dicho servidor web. Soporta usuarios virtuales mediante SQL, autenticacion LDAP y modulos personalizados.

```
sudo apt install proftpd -y
# Archivo de configuracion: /etc/proftpd/proftpd.conf
```

Pure-FTPd

Pure-FTPd es conocido por su simplicidad y alta seguridad. No tiene archivo de configuracion tradicional; en cambio, se configura mediante opciones en la linea de comandos o archivos separados. Soporta usuarios virtuales, cuotas de disco y cifrado TLS.

```
sudo apt install pure-ftpd -y
```

3. SERVIDOR WEB: IIS Y SU EQUIVALENTE EN UBUNTU (APACHE2)

3.1 Que es IIS y los servidores web en Linux

IIS (Internet Information Services) es el servidor web y de aplicaciones desarrollado por Microsoft para sus sistemas operativos Windows Server. Ofrece soporte para HTTP, HTTPS, FTP, SMTP y otros protocolos. Sin embargo, IIS no esta disponible de forma nativa para Linux ni Ubuntu Server.

En entornos Ubuntu Server, los equivalentes funcionales mas utilizados son:

- Apache HTTP Server (Apache2): el servidor web mas utilizado del mundo. Modular, estable y con soporte para multiples lenguajes de programacion.
- Nginx: servidor web y proxy inverso de alto rendimiento, ideal para sitios con alto trafico.
- Lighttpd: servidor web ligero orientado a entornos con recursos limitados.

En este apartado nos centraremos en Apache2, ya que es el equivalente mas directo a IIS en el ecosistema Linux y el mas utilizado en entornos educativos y de produccion.

3.2 Instalacion de Apache2 en Ubuntu Server

```
sudo apt update
sudo apt install apache2 -y
```

Verificamos el estado e iniciamos el servicio:

```
sudo systemctl status apache2
sudo systemctl enable apache2
sudo systemctl start apache2
```

Comprobamos que Apache escucha en el puerto 80:

```
sudo ss -tlnp | grep 80
```

Podemos verificar desde el navegador accediendo a `http://IP_DEL_SERVIDOR`. Si vemos la pagina de bienvenida de Apache2 Ubuntu Default Page, el servidor esta funcionando correctamente.

El firewall UFW debe permitir el trafico web:

```
sudo ufw allow 'Apache Full'    # permite HTTP (80) y HTTPS (443)
sudo ufw status
```

3.3 Estructura de directorios de Apache2

Ruta	Descripcion
/etc/apache2/	Directorio principal de configuracion
/etc/apache2/apache2.conf	Archivo de configuracion global
/etc/apache2/sites-available/	Definiciones de VirtualHosts disponibles
/etc/apache2/sites-enabled/	VirtualHosts activos (enlaces simbolicos)
/etc/apache2/mods-available/	Modulos disponibles
/etc/apache2/mods-enabled/	Modulos activos
/etc/apache2/conf-available/	Configuraciones adicionales disponibles
/etc/apache2/conf-enabled/	Configuraciones adicionales activas
/var/www/html/	Directorio raiz por defecto del servidor web
/var/log/apache2/	Logs de acceso y errores

3.4 Configuracion de VirtualHosts

Los VirtualHosts permiten alojar multiples sitios web en un mismo servidor fisico. Apache2 incluye una configuracion por defecto en /etc/apache2/sites-available/000-default.conf.

Crear un VirtualHost nuevo

```
sudo nano /etc/apache2/sites-available/miweb.conf
```

```
<VirtualHost *:80>
    ServerName www.miweb.com
    ServerAlias miweb.com
    ServerAdmin webmaster@miweb.com
    DocumentRoot /var/www/miweb
    ErrorLog ${APACHE_LOG_DIR}/miweb_error.log
    CustomLog ${APACHE_LOG_DIR}/miweb_access.log combined
</VirtualHost>
```

Creamos el directorio y habilitamos el sitio:

```
sudo mkdir -p /var/www/miweb
sudo chown -R www-data:www-data /var/www/miweb
sudo chmod -R 755 /var/www/miweb
echo '<h1>Bienvenido a miweb.com</h1>' | sudo tee
/var/www/miweb/index.html
sudo a2ensite miweb.conf
sudo a2dissite 000-default.conf # deshabilitar sitio por defecto
(opcional)
```

```
sudo systemctl reload apache2
```

3.5 Modulos de Apache2

Apache2 funciona mediante modulos que extienden su funcionalidad. Los comandos para gestionarlos son `a2enmod` (habilitar) y `a2dismod` (deshabilitar).

Modulo	Funcion
mod_rewrite	Reescritura de URLs (SEO, redirecciones)
mod_ssl	Soporte HTTPS/TLS
mod_headers	Manipulacion de cabeceras HTTP
mod_deflate	Compresion gzip de contenido
mod_expires	Control de cache del navegador
mod_auth_basic	Autenticacion HTTP basica
mod_proxy	Funciones de proxy inverso
mod_php	Soporte para PHP
mod_status	Pagina de estado del servidor
mod_security	Firewall de aplicaciones web (WAF)

```
# Habilitar modulos frecuentes
sudo a2enmod rewrite ssl headers deflate
sudo systemctl restart apache2
```

3.6 HTTPS con Let's Encrypt (Certbot)

HTTPS es imprescindible en cualquier sitio web en produccion. Let's Encrypt ofrece certificados SSL/TLS gratuitos y automatizados. En Ubuntu Server se usa la herramienta Certbot.

```
sudo apt install certbot python3-certbot-apache -y
sudo certbot --apache -d www.miweb.com -d miweb.com
```

Certbot modifica automaticamente la configuracion de Apache y renueva los certificados antes de que expiren. Para verificar la renovacion automatica:

```
sudo certbot renew --dry-run
```

Configuracion manual de HTTPS

```
<VirtualHost *:443>
    ServerName www.miweb.com
    DocumentRoot /var/www/miweb
    SSLEngine on
```

```

    SSLCertificateFile /etc/ssl/certs/miweb.crt
    SSLCertificateKeyFile /etc/ssl/private/miweb.key
    SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
    SSLCipherSuite HIGH:!aNULL:!MD5
    Header always set Strict-Transport-Security 'max-age=31536000'
</VirtualHost>

```

3.7 LAMP Stack: Apache + MySQL + PHP

Una de las configuraciones mas habituales en servidores Ubuntu es la pila LAMP (Linux, Apache, MySQL, PHP), que proporciona un entorno completo para aplicaciones web dinamicas.

```

# Instalar PHP
sudo apt install php libapache2-mod-php php-mysql -y
# Instalar MySQL
sudo apt install mysql-server -y
sudo mysql_secure_installation # configuracion segura inicial

```

Para comprobar que PHP funciona con Apache, crear un archivo de prueba:

```
echo '<?php phpinfo(); ?>' | sudo tee /var/www/html/info.php
```

Acceder a http://IP_DEL_SERVIDOR/info.php mostrara informacion completa sobre PHP.

3.8 Comparativa IIS vs Apache vs Nginx

Caracteristica	IIS	Apache2	Nginx
Sistema operativo	Windows	Linux/Windows/Mac	Linux/Windows/Mac
Coste	Incluido en Windows	Gratuito (open source)	Gratuito (open source)
Rendimiento alto trafico	Bueno	Bueno	Excelente
Consumo de memoria	Medio	Medio	Bajo
Configuracion	GUI + XML	Archivos .conf	Archivos .conf
VirtualHosts	Si (Sites)	Si	Si (Server Blocks)
Soporte PHP	Via FastCGI	mod_php / FastCGI	FastCGI
Modulos	Si	Si (extenso)	Limitado
Comunidad y soporte	Microsoft oficial	Muy amplia	Amplia

4. SERVICIO SSH (SECURE SHELL)

4.1 Que es SSH

SSH (Secure Shell) es un protocolo de red criptografico que permite la comunicacion segura entre dos sistemas a traves de una red insegura. Fue disenado como reemplazo seguro de protocolos inseguros como Telnet, rsh y rlogin, que transmitían datos en texto plano.

SSH proporciona tres garantias fundamentales de seguridad:

- Confidencialidad: toda la comunicacion esta cifrada, por lo que un atacante que intercepte el trafico no podra leer su contenido.
- Integridad: los datos no pueden ser modificados en transito sin que sea detectado.
- Autenticacion: tanto el cliente como el servidor verifican la identidad del otro antes de establecer la sesion.

SSH se utiliza principalmente para:

- Acceso remoto seguro a la linea de comandos de un servidor.
- Transferencia segura de archivos (SCP y SFTP).
- Tunelizacion y reenvio de puertos (port forwarding).
- Ejecucion remota de comandos y scripts.
- Montaje de sistemas de archivos remotos (SSHFS).

4.2 Historia y versiones del protocolo SSH

SSH fue creado en 1995 por Tatu Ylonen, investigador de la Universidad Tecnologica de Helsinki, tras sufrir un ataque de captura de contraseñas en su red universitaria. Creo la version 1.0 del protocolo y la herramienta openssh.

Version	Descripcion
SSH-1 (1995)	Primera version. Usa RSA para intercambio de claves. Tiene vulnerabilidades conocidas; NO debe usarse.
SSH-2 (1996-2006)	Version actual. Definida en RFC 4251-4256. Usa DH para intercambio de claves, soporta DSA, RSA y ECDSA. Es incompatible con SSH-1.
OpenSSH	Implementacion de codigo abierto mantenida por el proyecto OpenBSD. Es la implementacion mas utilizada en Linux y macOS.

4.3 Instalacion y activacion del servidor SSH

En Ubuntu Server, el servidor SSH (OpenSSH) puede estar preinstalado, pero si no lo esta:

```
sudo apt update
sudo apt install openssh-server -y
```

Gestion del servicio:

```
sudo systemctl status ssh
sudo systemctl enable ssh    # iniciar automaticamente en el arranque
sudo systemctl start ssh
sudo systemctl restart ssh   # aplicar cambios de configuracion
```

Verificar que SSH escucha en el puerto 22:

```
sudo ss -tlnp | grep 22
sudo netstat -tlnp | grep sshd
```

Permitir SSH en el firewall UFW:

```
sudo ufw allow ssh
sudo ufw allow 22/tcp
sudo ufw enable
sudo ufw status verbose
```

4.4 Configuración del servidor SSH

El archivo de configuración principal del servidor SSH es `/etc/ssh/sshd_config`. Es uno de los archivos más importantes a conocer para la administración de servidores Linux.

```
# /etc/ssh/sshd_config - Parametros principales
Port 22                                # puerto de escucha (cambiar por
seguridad)
AddressFamily inet                    # solo IPv4; usar 'any' para IPv4 e
IPv6
ListenAddress 0.0.0.0                # direccion en la que escucha

# Autenticacion
LoginGraceTime 2m                    # tiempo maximo para autenticarse
PermitRootLogin no                    # IMPORTANTE: deshabilitar login de
root
StrictModes yes                      # verificar permisos de archivos
MaxAuthTries 3                        # maximo de intentos de
autenticacion
MaxSessions 10                       # maximo de sesiones por conexion

# Autenticacion por clave publica
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

```
# Deshabilitar autenticacion por contrasena (recomendado con claves)
PasswordAuthentication yes          # cambiar a 'no' cuando se usen
claves
PermitEmptyPasswords no
ChallengeResponseAuthentication no

# Opciones de sesion
X11Forwarding no                    # deshabilitar si no se necesita
PrintLastLog yes                     # mostrar ultimo acceso al iniciar
sesion
TCPKeepAlive yes
ClientAliveInterval 300              # enviar keepalive cada 5 minutos
ClientAliveCountMax 2                # desconectar tras 2 keepalives sin
respuesta

# Restricciones de acceso
AllowUsers usuario1 usuario2         # solo estos usuarios pueden
conectar
DenyUsers usuariobloqueado
AllowGroups sshusers admins

# Subsistemas
Subsystem sftp /usr/lib/openssh/sftp-server
```

Despues de modificar la configuracion, verificar la sintaxis y reiniciar:

```
sudo sshd -t                        # verificar sintaxis sin reiniciar
sudo systemctl restart ssh
```

4.5 Autenticacion por clave publica/privada

La autenticacion por clave publica/privada es mucho mas segura que la autenticacion por contrasena. Se basa en criptografia asimetrica: el usuario posee una clave privada (secreta) y el servidor almacena la clave publica correspondiente.

Como funciona

1. El cliente genera un par de claves: publica y privada.
2. La clave publica se copia al servidor en ~/.ssh/authorized_keys.
3. Al conectar, el servidor envia un desafio cifrado con la clave publica del cliente.
4. Solo quien posee la clave privada puede descifrar el desafio y autenticarse.

Generacion del par de claves en el cliente

```
# En la maquina cliente (Linux/macOS)
ssh-keygen -t ed25519 -C 'mi_email@ejemplo.com'
```



```
# Para mayor compatibilidad con sistemas antiguos:
ssh-keygen -t rsa -b 4096 -C 'mi_email@ejemplo.com'
```

El comando genera dos archivos en ~/.ssh/:

- id_ed25519: clave PRIVADA. Nunca debe compartirse ni enviarse a nadie.
- id_ed25519.pub: clave PUBLICA. Es la que se copia al servidor.

Copiar la clave publica al servidor

```
# Metodo automatico (recomendado)
ssh-copy-id usuario@IP_SERVIDOR
# Metodo manual
cat ~/.ssh/id_ed25519.pub | ssh usuario@IP_SERVIDOR 'mkdir -p ~/.ssh
&& cat >> ~/.ssh/authorized_keys'
```

Permisos correctos en el servidor

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Conectar usando la clave

```
ssh usuario@IP_SERVIDOR
ssh -i ~/.ssh/mi_clave_especifica usuario@IP_SERVIDOR
```

4.6 Hardening y seguridad SSH

Asegurar correctamente el servicio SSH es fundamental. A continuacion se presentan las mejores practicas de seguridad:

1. Cambiar el puerto por defecto

Cambiar el puerto 22 reduce el ruido en los logs al evitar la mayoría de los ataques automatizados de fuerza bruta:

```
Port 2222      # o cualquier puerto alto no utilizado
sudo ufw allow 2222/tcp
sudo ufw delete allow 22/tcp
```

2. Deshabilitar el login de root

```
PermitRootLogin no
```

3. Usar solo autenticacion por clave

```
PasswordAuthentication no
PubkeyAuthentication yes
```

4. Limitar los usuarios con acceso SSH

```
AllowUsers usuario1 usuario2
```

5. Usar Fail2Ban para proteger contra ataques de fuerza bruta

Fail2Ban es una herramienta que bloquea automaticamente las IPs que generan demasiados intentos fallidos de autentificacion.

```
sudo apt install fail2ban -y
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

```
[sshd]
enabled = true
port    = ssh
filter  = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600      # ban de 1 hora
findtime = 600      # ventana de 10 minutos
```

```
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
sudo fail2ban-client status sshd  # ver IPs baneadas
```

6. Configurar el algoritmo de cifrado

Especificar solo algoritmos fuertes y modernos:

```
KexAlgorithms curve25519-sha256,diffie-hellman-group14-sha256
Ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com
MACs hmac-sha2-256,hmac-sha2-512
```

4.7 Tunel SSH y reenvio de puertos

El reenvio de puertos (port forwarding) es una de las funcionalidades mas poderosas de SSH. Permite redirigir trafico de red a traves de una conexion SSH cifrada.

Reenvio local (Local Port Forwarding)

Permite acceder a un servicio en el servidor remoto (o en su red) como si estuviera en nuestra maquina local:

```
ssh -L puerto_local:host_destino:puerto_destino usuario@servidor
# Ejemplo: acceder a MySQL remoto como si fuera local
```

```
ssh -L 3307:localhost:3306 admin@192.168.1.100
mysql -h 127.0.0.1 -P 3307 -u root -p
```

Reenvio remoto (Remote Port Forwarding)

Expone un servicio local a traves del servidor remoto. Util para acceder a servicios detras de NAT:

```
ssh -R puerto_remoto:localhost:puerto_local usuario@servidor
# Ejemplo: exponer servidor web local en el servidor remoto
ssh -R 8080:localhost:80 admin@servidor.com
```

Reenvio dinamico (Dynamic Port Forwarding - SOCKS proxy)

Crea un proxy SOCKS que puede usarse para enrutar trafico de aplicaciones a traves del servidor SSH:

```
ssh -D 1080 usuario@servidor
# Configurar el navegador para usar proxy SOCKS5 en localhost:1080
```

4.8 SCP y SFTP: transferencia segura de archivos

SCP (Secure Copy Protocol)

SCP permite copiar archivos entre sistemas de forma segura usando SSH. Es sencillo y rapido para transferencias puntuales.

```
# Copiar archivo local al servidor
scp archivo.txt usuario@servidor:/ruta/destino/
# Copiar archivo del servidor a local
scp usuario@servidor:/ruta/archivo.txt ./
# Copiar directorio completo (recursivo)
scp -r directorio/ usuario@servidor:/ruta/destino/
# Usar puerto SSH personalizado
scp -P 2222 archivo.txt usuario@servidor:/ruta/
```

SFTP (SSH File Transfer Protocol)

SFTP es un protocolo de transferencia de archivos que funciona sobre SSH. A diferencia de SCP, ofrece una sesion interactiva similar a FTP pero completamente cifrada.

```
sftp usuario@servidor
sftp> ls          # listar archivos remotos
sftp> ll          # listar archivos locales
sftp> get archivo.txt # descargar archivo
sftp> put archivo.txt # subir archivo
sftp> mkdir nueva_carpeta
sftp> exit
```

SFTP puede restringirse a ciertos usuarios mediante la directiva Match en sshd_config, útil para crear usuarios que solo puedan transferir archivos:

```
Match User sftpuser
    ChrootDirectory /home/sftpuser
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
```

5. COMPARATIVA DE SEGURIDAD ENTRE SERVICIOS

La siguiente tabla compara los aspectos de seguridad mas relevantes de los tres servicios estudiados:

Aspecto	FTP (vsftpd)	Apache2 (Web)	SSH (OpenSSH)
Cifrado por defecto	No (texto plano)	No (HTTP) / Si (HTTPS)	Si (siempre)
Puerto por defecto	21 (control), 20 (datos)	80 (HTTP), 443 (HTTPS)	22
Autenticacion	Usuario/contrasena	Anonimo / HTTP Auth / Cert.	Contrasena / Clave publica
Riesgo de intercepc.	Alto sin TLS	Alto sin HTTPS	Muy bajo
Version segura	FTPS (RFC 4217)	HTTPS + TLS 1.2/1.3	SSH-2 unicamente
Proteccion fuerza bruta	Fail2Ban / limites	mod_security / Fail2Ban	Fail2Ban / MaxAuthTries
Recomendado en prod.	Solo con FTPS o SFTP	Con HTTPS obligatorio	Si, con clave publica

6. BUENAS PRACTICAS Y RECOMENDACIONES

6.1 Buenas practicas generales

- Mantener siempre el sistema operativo y los paquetes actualizados: `sudo apt update && sudo apt upgrade -y`
- Revisar regularmente los logs del sistema: `/var/log/syslog`, `/var/log/auth.log`, `/var/log/apache2/`, `/var/log/vsftpd.log`
- Usar el principio de minimo privilegio: cada usuario y servicio debe tener solo los permisos estrictamente necesarios.
- Configurar el firewall UFW correctamente, permitiendo solo los puertos necesarios.
- Realizar copias de seguridad periodicas de la configuracion y los datos.
- Usar contraseñas fuertes o mejor aun, autenticacion por clave publica para SSH.
- Deshabilitar los servicios que no se usen.

6.2 Buenas practicas especificas por servicio

FTP

- Usar siempre FTPS o sustituir FTP por SFTP.
- Deshabilitar el acceso anonimo.
- Encerrar a los usuarios en su directorio (chroot).
- Restringir el rango de puertos pasivos y abrir solo esos en el firewall.

Apache2 / Web

- Forzar redireccion de HTTP a HTTPS.
- Mantener los modulos innecesarios deshabilitados.
- Configurar cabeceras de seguridad: HSTS, X-Frame-Options, X-Content-Type-Options, CSP.
- Mantener PHP, WordPress y demas aplicaciones actualizadas.
- Usar mod_security como WAF (Web Application Firewall).

SSH

- Cambiar el puerto por defecto 22 por uno distinto.
- Deshabilitar el acceso de root por SSH.
- Usar exclusivamente autentificacion por clave publica en produccion.
- Instalar y configurar Fail2Ban.
- Usar solo SSH-2 y algoritmos modernos.
- Revisar periodicamente los usuarios en authorized_keys.

7. REFERENCIAS Y BIBLIOGRAFIA

A continuacion se listan las referencias tecnicas y normativas utilizadas para la elaboracion de este documento:

- RFC 959 (1985) - Postel, J. y Reynolds, J.: 'File Transfer Protocol'. IETF.
- RFC 4217 (2007) - Ford-Hutchinson, P.: 'Securing FTP with TLS'. IETF.
- RFC 4251 (2006) - Ylonen, T. y Lonvick, C.: 'The Secure Shell (SSH) Protocol Architecture'. IETF.
- RFC 4252 - 'The Secure Shell (SSH) Authentication Protocol'. IETF.
- RFC 4253 - 'The Secure Shell (SSH) Transport Layer Protocol'. IETF.
- Documentacion oficial de Ubuntu Server: <https://ubuntu.com/server/docs>
- Documentacion oficial de OpenSSH: <https://www.openssh.com/manual.html>

- Documentacion oficial de Apache HTTP Server:
<https://httpd.apache.org/docs/>
- Pagina del proyecto vsftpd: <https://security.appspot.com/vsftpd.html>
- CIS Benchmarks para Ubuntu Linux: <https://www.cisecurity.org/>
- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment.
- Documentacion de Fail2Ban: <https://www.fail2ban.org/wiki/>

Nota: Toda la informacion de configuracion presentada en este documento corresponde a Ubuntu Server 22.04 LTS (Jammy Jellyfish) con los paquetes disponibles en los repositorios oficiales de Ubuntu a fecha de 2024.